Detailed Data Preprocessing Pipeline for Monthly Return Prediction

1 Overview

We organize the data preprocessing pipeline for a supervised learning problem on monthly equity returns. The input data are panel data, meaning that each observation belongs to one entity observed repeatedly over time. In this project, the entity is a company and the time index is a month. Each row therefore represents one company–month pair, and the central prediction target is the variable `monthly_gross_return`, which records the realized return for that company in that month.

The preprocessing goal is to transform raw company–month records into a leakage-safe modeling table that can be consumed by two downstream learners: CatBoost, which learns tabular cross-sectional structure, and a convolutional neural network, which learns residual patterns from short historical sequences. The preprocessing stage must therefore do more than clean columns. It must also enforce time ordering, prevent future information from entering past feature values, standardize categorical fields, construct lagged and rolling features, label missing values explicitly, and prepare split-specific training frames.

At a high level, we want to find a mapping from current and past company characteristics to next observed monthly return behavior. In other words, we want to learn how the state of a company at month t, together with its recent history, relates to its return at that same month. The preprocessing pipeline exists to make that mapping learnable while preserving the causal order of the panel.

2 Dataset Description

The dataset is stored as three CSV files:

- `processed_data/train.csv`
- `processed_data/validation.csv`
- `processed_data/test.csv`

These files represent a chronological split of the same underlying panel. The raw records are company–month observations, and the important columns fall into four groups.

2.1 Target Variable

The target is `monthly_gross_return`. This is the quantity we seek to predict. Throughout the pipeline, any model output is compared against this target to compute residuals and evaluation metrics.

2.2 Entity and Time Variables

The entity identifier is `co_code`. We define the entity as a company, and we treat `co_code` as the stable identifier that groups all months belonging to the same company.

The time variable is `Month`. It is converted to a datetime object and used as the chronological index. The dataset also contains supporting calendar fields such as `Year`, `Corrected_Year`, and `Corrected_Month`. These are not the primary ordering keys in the pipeline, because the preprocessing logic relies on the full month timestamp rather than on separate year and month integers.

2.3 Numerical Features

The raw data include numerical company characteristics and market variables such as `Momentum`, `log_ret`, `mktcap`, `price`, `BM_sep`, `OpProf`, `Inv`, and related lagged or derived quantities such as `lag_mv`, `lagged_assets`, and `lagged_book_equity`. We define a numerical feature as a variable with arithmetic meaning, so operations such as subtraction, rolling averages, or standardization are valid on it.

2.4 Categorical Features

The categorical features are `Size_Label`, `BM_Label`, `OpProf_Label`, `Inv_Label`, and `Mom_Label`. We define a categorical feature as a discrete label whose values represent groups or buckets rather than magnitudes. These columns are passed explicitly to CatBoost and are encoded carefully for the CNN stage.

2.5 Why This Dataset Is a Panel

We call the dataset a panel because the same company may appear in many months, and the same month contains many companies. This creates two natural axes:

- a cross-sectional axis across companies within a month,
- a temporal axis across months for each company.

The preprocessing pipeline must preserve both axes. Cross-sectional structure matters because companies differ from one another in size, value, profitability, investment, momentum, and missingness patterns. Temporal structure matters because return dynamics depend on recent history, not only on the current row.

3 What We Want to Find

The objective is to estimate monthly gross return from structured company information. More specifically, we want to find two complementary signal components:

1. a tabular signal, which can be learned from the current row and its engineered summary features,
2. a residual temporal signal, which remains after the tabular model has explained the obvious part of the return.

This division is important because financial panel data usually contain both stable cross-sectional effects and weaker but persistent temporal effects. CatBoost is used to extract the first component. A convolutional network is then trained on the remaining error, which we define as the residual. The preprocessing pipeline must therefore produce a clean feature matrix for CatBoost and a sequence tensor for the convolutional network.

4 Workflow Organization

We organize the full preprocessing and modeling workflow into the following stages:

1. load the three split files,
2. concatenate them temporarily for feature engineering,
3. standardize the column types,
4. remove duplicate company–month rows,
5. build lagged, rolling, and missingness features,
6. reconstruct the split frames,
7. train CatBoost on the train split,
8. compute out-of-sample residuals on validation and test,
9. build CNN training sequences from the residual stage,
10. train the residual CNN,
11. evaluate both models and the combined prediction.

The key rule is that temporary concatenation is used only for feature engineering. It is never used to let information from future months leak backward into past feature construction.

5 Data Loading and Initial Assembly

We first load `train.csv`, `validation.csv`, and `test.csv`. Each file is tagged with a split label so that the source of each row remains visible after concatenation. The pipeline uses a dedicated column, `source_split`, to store the origin of every row.

The three split files are concatenated into one panel frame. The temporary concatenation serves one purpose: it allows us to build lags and rolling statistics consistently across split boundaries. Without this temporary merge, a company whose historical window straddles the train–validation boundary would have an incomplete feature history.

After feature construction, the combined frame is split back into the original train, validation, and test subsets using `source_split`. This means that feature generation may see the full chronological history, but model fitting remains split-aware.

6 Type Coercion and Canonical Sorting

Before any feature engineering begins, we standardize the core column types.

6.1 Datetime Conversion

We convert `Month` to a datetime object. This conversion is essential because the month field is later used to sort rows, define lagged windows, and derive calendar-based features such as month indices.

6.2 Identifier Conversion

We convert `co_code` to a numeric identifier with nullable integer support. The identifier is not treated as a feature to be learned by the model. Instead, it is used as a grouping key for all company-level temporal operations.

6.3 Target Conversion

We convert `monthly_gross_return` to a numeric field. This ensures that all downstream loss computations, residual calculations, and metric formulas operate on a consistent numeric representation.

6.4 Categorical Normalization

For each categorical label column, we convert values to strings, strip whitespace, replace empty or malformed text with missing markers, and then fill missing values with the explicit token `__MISSING__`. This normalization prevents mixed-type warnings and ensures that the same categorical vocabulary is used across the entire pipeline.

The need for normalization arises because categorical data in raw CSV files may contain a mix of numeric codes, strings, empty entries, and missing values. If left untreated, these inconsistencies can cause unstable encoding behavior and inconsistent category maps.

6.5 Numeric Conversion of Remaining Columns

All columns that are not the entity identifier, the target, the date, the split label, or one of the known categorical columns are converted to numeric values with coercion. Any non-numeric entries become missing values. This policy is deliberate. We prefer to preserve uncertainty as missingness rather than silently invent numeric values.

6.6 Canonical Sort Order

Once the types are standardized, we sort the panel by `co_code` and `Month` using a stable sorting algorithm. The stable sort is important because it preserves the relative order of duplicate keys until they are intentionally collapsed. Canonical sorting ensures that all time-based operations are applied in the correct chronological order within each company.

7 Duplicate Company–Month Handling

In a clean panel, each company should appear at most once per month. A duplicate company–month row is a repeated observation with the same `co_code` and `Month`. Duplicate rows are problematic because they distort lag construction, inflate sample counts, and create ambiguity about the correct target value for a given company-month key.

We detect duplicates using the pair `(co_code, Month)`. When duplicates are present, we collapse them by keeping the last occurrence for each key. This rule is explicit and deterministic. It provides one canonical row per company-month pair and removes ambiguity before feature engineering begins.

We also record how many duplicate keys were present and how many rows were removed. This diagnostic matters because duplicate collapse is a data quality event, not a silent housekeeping step. If duplicate rows exist, they are a sign that the underlying source files contained overlapping or repeated records.

8 Feature Engineering

Feature engineering transforms raw panel columns into model-ready inputs. In this project, feature engineering is entirely leakage-safe. We define leakage as any use of future information when forming features for a current row. Leakage is forbidden because it creates unrealistic validation scores and invalidates the trained model.

8.1 Lag Features

We construct lag features for the target return using the past values of the same company. A lag is a shifted version of a time series. If $y(j,t)$ is the return of company j at month t, then a lag feature at lag k is the value observed at month $t-k$.

The implemented lag steps are 1, 3, and 6 months. These become features such as:

- `lag_ret_1`,
- `lag_ret_3`,
- `lag_ret_6`.

We also compute lagged versions of `log_ret` when that column is available. This is useful because logarithmic returns often contain different local structure from gross returns.

The lag feature family captures short-term persistence and reversal effects. For example, if a company experienced a strong return one month ago, the current month may partially reflect momentum or mean reversion. The lag columns allow CatBoost and the CNN to learn those patterns directly.

8.2 Rolling Features

We compute rolling statistics over the past history of each company. A rolling statistic is a summary computed over a trailing window of past observations. The windows used here are 3 months and 6 months.

For each company and each window, we compute:

- rolling mean of the past return history,
- rolling standard deviation of the past return history.

These become features such as `rolling_mean_3`, `rolling_std_3`, `rolling_mean_6`, and `rolling_std_6`. The rolling mean summarizes average recent performance, while the rolling standard deviation summarizes recent volatility.

The computation uses only past observations. We first shift the target by one month so that the current month is excluded from the rolling window. This makes the rolling statistics causal, meaning they depend only on information that would have been available at the time of prediction.

8.3 Month Index

We derive a calendar feature called `month_index`. This is computed from the year and month components of `Month`. The month index acts as a monotonic time marker. It is useful for sorting, diagnostics, and future extensions where calendar progression may matter as a feature.

8.4 Missingness Indicators

For every column that is not an identifier or a split label, we add a binary missingness indicator of the form `is_missing_{feature}`. A missingness indicator is a 0/1 feature that records whether the original field was missing.

This step is important because missing values in financial data are often informative. A missing value may indicate a reporting regime, a liquidity threshold, a data availability issue, or a structural difference between firms. If we only leave the missing entry as `NaN`, the model may still handle it, but the explicit indicator allows the model to learn whether absence itself carries predictive signal.

8.5 Drop Previously Engineered Columns

Because the preprocessing pipeline may be rerun multiple times, we first remove previously engineered columns if they already exist in the input files. These include lag features, rolling features, missing indicators, and older engineered fields such as `split_group` or prior residual columns. This prevents feature duplication and avoids accidental nesting of engineered variables.

8.6 Cross-Sectional Features

We keep raw company characteristics such as `mktcap`, `BM_sep`, `OpProf`, and `Inv`. These are cross-sectional features because they describe the state of a company in a given month rather than a time-series summary over many months. They help the model distinguish company size, value, profitability, investment intensity, and other structural effects.

8.7 Categorical Features for CatBoost

The label columns `Size_Label`, `BM_Label`, `OpProf_Label`, `Inv_Label`, and `Mom_Label` are preserved as explicit categorical variables. CatBoost receives them as categorical inputs rather than as numeric magnitudes. This matters because the labels represent ordered or bucketed groups whose encoded integers should not be interpreted as continuous values by the model.

9 Feature Set Construction

After feature engineering, we define the model input set. The CatBoost feature table includes all columns except:

- the target `monthly_gross_return`,
- the company identifier `co_code`,
- the date field `Month`,
- the split label `source_split`,
- internal bookkeeping columns such as `row_id`,
- intermediate model outputs such as `cb_pred`, `cnn_resid_pred`, and `final_pred`.

We call this exclusion list a feature filter. A feature filter is the set of columns we intentionally remove from the learning table because they are identifiers, labels, targets, or internal outputs rather than genuine predictive inputs.

The remaining columns become the feature columns for CatBoost. The categorical subset is identified by selecting the known label columns that remain in the feature list.

10 Split Verification

Once the split frames are reconstructed, we verify that the train, validation, and test periods are strictly chronological. The expected condition is:

- the last month in train is earlier than the first month in validation,
- the last month in validation is earlier than the first month in test.

This verification step protects against accidental overlap between splits. In time series and panel data, a random split would be invalid because the model could be trained on future months and then evaluated on earlier ones. The chronological test ensures that each validation or test row is genuinely out of sample.

We also re-check that duplicate company–month rows have been removed. If any remain, the pipeline re-collapses them before continuing. This makes the split verification stage both a chronology check and a structural integrity check.

11 CatBoost-Specific Preprocessing

CatBoost is trained on the cleaned train split only. To build its training matrix, we reindex each frame to the feature column list and convert categorical columns to strings with the explicit missing token. This ensures that train, validation, and test feature matrices have identical column order and compatible categorical representations.

CatBoost benefits from several properties of the preprocessing pipeline:

- missing values are preserved as missing rather than imputed globally,
- categorical labels remain categorical,
- lagged and rolling features encode temporal dependence,
- duplicate company–month keys are removed,
- all features are aligned across the same model vocabulary.

The CatBoost model itself produces the first-stage prediction, which we denote as $\hat{y}_{\mathrm{CB}}(j,t)$. The preprocessing pipeline must make this prediction possible by defining a consistent feature table.

12 Residual Construction

The residual is the difference between the observed return and the CatBoost prediction. If $y(j,t)$ is the actual monthly return and $\hat{y}_{\mathrm{CB}}(j,t)$ is the CatBoost output, then the residual is

$$e(j,t) = y(j,t) - \hat{y}_{\mathrm{CB}}(j,t).$$

We define a residual as the part of the target not explained by the first-stage model. The residual is critical because the CNN is trained to learn this remaining structure, not the original target directly.

Residuals are computed only on out-of-sample rows. In this pipeline, that means validation and test rows receive CatBoost predictions, and the difference between actual and predicted values becomes the residual target for the second stage. Training residuals are not used as direct supervision because in-sample residuals would be too optimistic and would leak fitting behavior into the CNN stage.

13 CNN Sequence Construction

The convolutional network does not consume single rows only. Instead, it consumes short sequences of past feature vectors. We define a sequence as a fixed-length window of consecutive company-month feature vectors.

If the sequence length is L, then for company j at month t the CNN input is a tensor containing the feature vectors from months $t-L+1$ through t. This tensor is arranged in channel-first format so that the convolution operates across the time dimension.

13.1 Why Sequence Construction Is Needed

The CNN is designed to exploit local temporal dependence. A single row can describe the state of a firm at one month, but a short sequence can describe how that state evolved across recent months. This matters because residual structure may depend on recent patterns rather than only on contemporaneous values.

13.2 Causal Windows

We define a causal window as a sequence that uses only current and past observations. Future observations are excluded. Causality matters because the model must be usable in live inference, where future months do not exist yet.

In the implemented pipeline, the sequence length is six months. For each eligible row, we take the six most recent company-month feature vectors, normalize them with statistics estimated only from the designated training portion, and transpose the result so that the CNN sees features as channels.

13.3 Feature Normalization for Sequences

Before sequence training, we compute a mean and standard deviation for the CNN feature space using the rows designated for sequence-model training. We then standardize each window using those statistics. Standardization means subtracting the mean and dividing by the standard deviation.

This step is necessary because the CNN is sensitive to scale. Without standardization, large-magnitude features such as market capitalization could dominate the convolutional filters, while smaller features could become numerically invisible.

13.4 CNN Feature Augmentation

The CNN input includes the CatBoost prediction as an additional channel. We define this augmentation as adding the first-stage prediction to the sequence feature set. The idea is that the second-stage network should have access to the first-stage forecast when learning how to correct it.

14 Validation and Test Sequence Split

The validation period is further divided into a CNN training portion and a CNN holdout portion. This is a second-level chronological split inside the validation set. We use it because the CNN must also be trained and checked in an out-of-sample way.

The split is performed month by month. Early validation months are assigned to CNN training, and later validation months are reserved as CNN holdout. This keeps the residual learner from seeing the entire validation horizon during fitting.

The test set remains untouched until final evaluation. Only rows with sufficient history to form a full sequence are eligible for CNN scoring. If a row does not have enough past months to fill the sequence length, the CNN cannot produce a residual correction for that row. In that case, the final prediction falls back to the CatBoost output alone.

15 Model-Ready Artifacts

The preprocessing pipeline produces the following artifacts:

- the cleaned combined panel,
- the split-specific frames for train, validation, and test,
- the CatBoost feature list,
- the list of categorical features,
- the CatBoost prediction column,
- the base residual column,
- the CNN feature matrix,
- the sequence tensors for CNN training, validation, and test,
- the final combined prediction column.

We call these artifacts because they are the structured outputs of preprocessing rather than final business results. Each artifact supports a different downstream stage.

16 Inference Preparation

For inference, we append new rows to the historical panel, rebuild features, and score them in chronological order. The historical frame contains all past company-month rows, and the new frame contains the rows to be predicted. We label the historical rows as history and the new rows as inference so that sequence generation can restrict attention to the rows that require predictions.

The inference procedure repeats the same preprocessing logic as training:

1. concatenate history and new rows,
2. drop older engineered columns,
3. coerce types,
4. collapse duplicate company–month rows,
5. rebuild lag and rolling features,
6. add missingness indicators,
7. sort by company and month,
8. compute CatBoost predictions,

9. encode the sequence features for the CNN,
10. construct causal windows for eligible new rows,
11. generate residual corrections,
12. add the residual correction to the CatBoost prediction.

The final output includes the CatBoost prediction, the CNN residual prediction, a flag indicating whether a sequence was available, and the combined final prediction.

17 Definitions of Recursively Introduced Terms

To keep the pipeline fully explicit, we restate the main terms in the order they are used.

Panel data data in which the same entity is observed repeatedly over time.

Entity the unit of observation repeated across months; here, a company.

Time index the ordered temporal coordinate; here, `Month`.

Feature a model input derived from the raw data or from a deterministic transformation of the raw data.

Leakage the use of information from the future when building a feature for the present.

Lag a shifted value from an earlier month.

Rolling statistic a summary computed over a trailing historical window.

Missingness indicator a binary variable that records whether another field is missing.

Residual the difference between the true target and the first-stage prediction.

Sequence tensor a stack of past feature vectors arranged as an input to a convolutional model.

Causal window a sequence that uses only current and earlier months.

Split integrity the guarantee that train, validation, and test occupy separate time ranges.

18 Final Summary

The preprocessing pipeline transforms raw monthly company data into a structured learning problem that respects time, prevents leakage, and supports a two-stage model. First, we prepare a leakage-safe tabular feature table for CatBoost. Second, we compute out-of-sample residuals and build temporal sequences for a CNN that learns the remaining error structure. Every major step is designed around the same principle: use only information that would have been available at prediction time, keep the panel ordered by company and month, and preserve enough structure for both cross-sectional and temporal learning.

In concrete terms, we start from a dataset of company–month rows, define the prediction target, standardize types, remove duplicates, engineer lagged and rolling features, add missingness indicators, maintain chronological splits, and prepare model-specific inputs for the residual-learning pipeline. The result is a complete preprocessing chain that supports reliable training, valid evaluation, and chronological inference.